

OBJECT ORIENTED PROGRAMING

OOP

PRACTICALS

NAME: SYEDA FAIZA ASLAM

SECTION: B

ROLL NO: CT-25064

LAB NO: 07

COURSE CODE: CT-260

COURSE TEACHER: Mr. AHMED ZAKI

Exercise

QUESTION: 1

Create a class for a **two-dimensional dynamic array (2D Array)** of type integer using a **double pointer** (i.e., a pointer to an array of pointers).

The class should implement the following member functions:

- Default constructor
- Parameterized constructor
- Copy constructor
- Overloaded assignment operator (=)
- Overloaded arithmetic operators:
 - Addition (+)
 - Subtraction (-)
 - Multiplication (*)
- Overloaded indexing operator ([])
- Destructor

Finally, write a **test program (main function)** to verify the correct working of the designed class.

SOURCECODE:

```
#include <iostream>
using namespace std;
```

```
class Array2D {
private:
    int **arr;
    int rows, cols;
```

```
public:
```

```
    // Default
    Constructor
    Array2D() {
```

```
    rows = 0;
    cols = 0;
    arr = nullptr;
}
```

```
// Parameterized
Constructor
Array2D(int r, int c) {
    rows = r;
    cols = c;
```

```
    arr = new
int*[rows];
    for (int i = 0; i <
rows; i++) {
        arr[i] = new
int[cols];
    }
}
```

```
// Copy Constructor
(Deep Copy)
Array2D(const
Array2D &obj) {
    rows = obj.rows;
    cols = obj.cols;
```

```
    arr = new
int*[rows];
    for (int i = 0; i <
rows; i++) {
        arr[i] = new
int[cols];
        for (int j = 0; j <
cols; j++) {
```

```

        arr[i][j] =
obj.arr[i][j];
    }
}
}

```

```

// Assignment
Operator Overloading
Array2D&
operator=(const
Array2D &obj) {
    if (this != &obj) {

```

```

        // Delete old
memory
        for (int i = 0; i <
rows; i++) {
            delete[] arr[i];
        }
        delete[] arr;

```

```

        // Allocate new
memory
        rows = obj.rows;
        cols = obj.cols;

```

```

        arr = new
int*[rows];
        for (int i = 0; i <
rows; i++) {
            arr[i] = new
int[cols];
            for (int j = 0; j
< cols; j++) {
                arr[i][j] =
obj.arr[i][j];

```

```

    }
}
return *this;
}

```

// Indexing Operator
Overloading

```

int* operator[](int i)
{
    return arr[i];
}

```

// Addition Operator

```

Array2D
operator+(const
Array2D &obj) {
    Array2D
temp(rows, cols);

```

```

    for (int i = 0; i <
rows; i++) {
        for (int j = 0; j <
cols; j++) {
            temp.arr[i][j]
= arr[i][j] + obj.arr[i][j];
        }
    }
    return temp;
}

```

// Subtraction
Operator

```

Array2D operator-
(const Array2D &obj) {

```

```

        Array2D
temp(rows, cols);

        for (int i = 0; i <
rows; i++) {
            for (int j = 0; j <
cols; j++) {
                temp.arr[i][j]
= arr[i][j] - obj.arr[i][j];
            }
        }
        return temp;
    }

```

```

// Multiplication
Operator (Matrix
Multiplication)
    Array2D
operator*(const
Array2D &obj) {
    Array2D
temp(rows, obj.cols);

        for (int i = 0; i <
rows; i++) {
            for (int j = 0; j <
obj.cols; j++) {
                temp.arr[i][j]
= 0;
                for (int k = 0;
k < cols; k++) {

temp.arr[i][j] +=
arr[i][k] * obj.arr[k][j];
                }
            }
        }
    }

```

```
    }  
    return temp;  
}
```

```
// Input Function  
void input() {  
    for (int i = 0; i <  
rows; i++) {  
        for (int j = 0; j <  
cols; j++) {  
            cin >> arr[i][j];  
        }  
    }  
}
```

```
// Display Function  
void display() {  
    for (int i = 0; i <  
rows; i++) {  
        for (int j = 0; j <  
cols; j++) {  
            cout <<  
arr[i][j] << " ";  
        }  
        cout << endl;  
    }  
}
```

```
// Destructor  
~Array2D() {  
    for (int i = 0; i <  
rows; i++) {  
        delete[] arr[i];  
    }  
    delete[] arr;  
}
```

```
};
```

```
// Test Program (Main  
Function)
```

```
int main() {
```

```
    int r, c;
```

```
    cout << "Enter rows  
and columns: ";
```

```
    cin >> r >> c;
```

```
    Array2D A(r, c), B(r,  
c);
```

```
    cout << "Enter  
elements of Matrix  
A:\n";
```

```
    A.input();
```

```
    cout << "Enter  
elements of Matrix  
B:\n";
```

```
    B.input();
```

```
    cout << "\nMatrix  
A:\n";
```

```
    A.display();
```

```
    cout << "\nMatrix  
B:\n";
```

```
    B.display();
```

```
// Addition
```

```
Array2D C = A + B;
```



```
cout << "\nA + B:\n";  
C.display();
```

```
// Subtraction  
Array2D D = A - B;  
cout << "\nA - B:\n";  
D.display();
```

```
// Multiplication  
(only if valid)  
if (c == r) {  
    Array2D E = A * B;  
    cout << "\nA *  
B:\n";  
    E.display();  
} else {  
    cout <<  
"\nMultiplication not  
possible (cols of A !=  
rows of B)\n";  
}  
  
return 0;  
}
```

OUTPUT:

```
C:\Users\AA\Desktop\OOP lab 7 (1).exe
Enter rows and columns: 2
3
Enter elements of Matrix A:
2
1
5
4
6
3
Enter elements of Matrix B:
4
5
2
1
3
4

Matrix A:
2 1 5
4 6 3

Matrix B:
4 5 2
1 3 4

A + B:
6 6 7
5 9 7

A - B:
-2 -4 3
3 3 -1

Multiplication not possible (cols of A != rows of B)
```

QUESTION: 2

Using the **2D Array class** created in Question 1, solve the following problem:

Given a **2D integer matrix of size $m \times n$** with the following properties:

- Each row is sorted in **non-decreasing order**
- The first element of each row is greater than the last element of the previous row

And given an integer **target**, determine whether the target exists in the matrix or not.

You must apply the **Binary Search algorithm** to solve this problem efficiently.

Input Example:

Matrix =

[[1, 3, 5, 7], [10, 11, 16, 20], [23, 30, 34, 60]]

Target = 3

Output:

True

SOURCECODE:

```
#include <iostream>
using namespace std;
```

```
// Function to display
matrix
void displayMatrix(int
**matrix, int rows, int
cols) {
    cout << "\nMatrix
is:\n";
    for (int i = 0; i <
rows; i++) {
        for (int j = 0; j <
cols; j++) {
            cout <<
matrix[i][j] << " ";
        }
        cout << endl;
    }
}
```

```
// Binary Search
Function
bool searchMatrix(int
**matrix, int rows, int
cols, int target) {
    int left = 0;
    int right = rows *
cols - 1;

    while (left <= right) {
        int mid = (left +
right) / 2;
```

```

        int r = mid / cols;
// row index
        int c = mid % cols;
// column index

        if (matrix[r][c] ==
target)
            return true;
        else if (matrix[r][c]
< target)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return false;
}

```

```

int main() {
    int rows, cols;

    // Step 1: Input size
    cout << "Enter
number of rows: ";
    cin >> rows;

    cout << "Enter
number of columns: ";
    cin >> cols;

    // Step 2: Dynamic
Memory Allocation
    int **matrix = new
int*[rows];
    for (int i = 0; i <
rows; i++) {

```

```
        matrix[i] = new  
int[cols];  
    }
```

```
    // Step 3: Input  
elements  
    cout << "\nEnter  
matrix elements (row-  
wise sorted):\n";  
    for (int i = 0; i <  
rows; i++) {  
        for (int j = 0; j <  
cols; j++) {  
            cin >>  
matrix[i][j];  
        }  
    }
```

```
    // Step 4: Display  
matrix
```

```
displayMatrix(matrix,  
rows, cols);
```

```
    // Step 5: Input  
target  
    int target;  
    cout << "\nEnter  
target value: ";  
    cin >> target;
```

```
    // Step 6: Search  
if  
(searchMatrix(matrix,  
rows, cols, target))
```

```

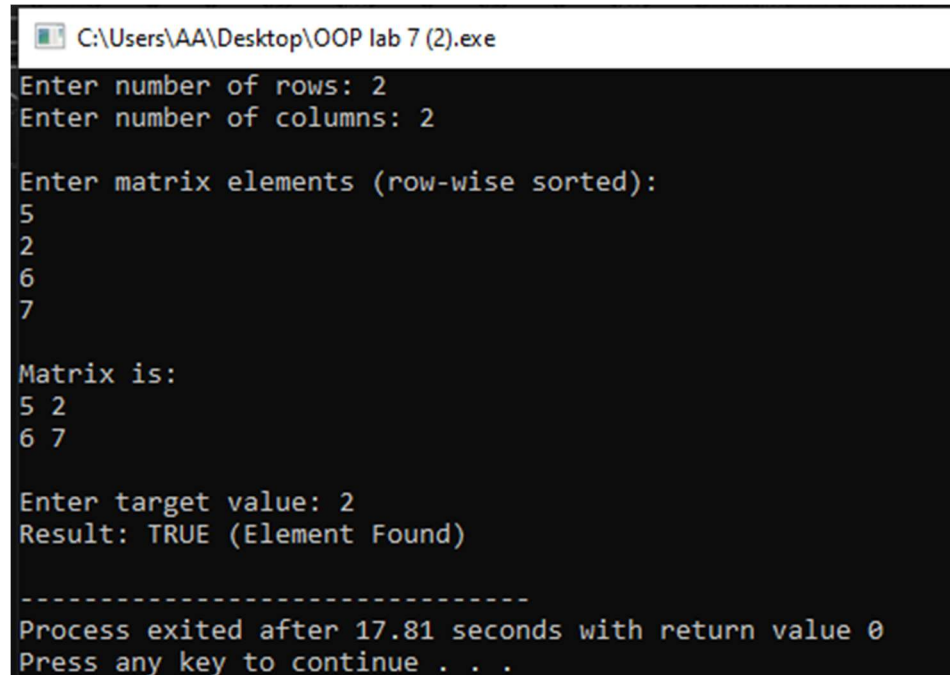
        cout << "Result:
TRUE (Element
Found)\n";
    else
        cout << "Result:
FALSE (Element Not
Found)\n";

    // Step 7: Free
memory
    for (int i = 0; i <
rows; i++) {
        delete[] matrix[i];
    }
    delete[] matrix;

    return 0;
}

```

OUTPUT:



```

C:\Users\AA\Desktop\OOP lab 7 (2).exe
Enter number of rows: 2
Enter number of columns: 2

Enter matrix elements (row-wise sorted):
5
2
6
7

Matrix is:
5 2
6 7

Enter target value: 2
Result: TRUE (Element Found)

-----
Process exited after 17.81 seconds with return value 0
Press any key to continue . . .

```

QUESTION: 3

A company wants to update employee salaries in its payroll system.

Implement the concept of a **friend class** using the following requirement:

- Create a class **Employee** with private data members:
 - name
 - id
 - designation
 - salary
- Create another class **Payroll**.

The **Payroll class** should contain a function to update the salary of an employee.

Since salary is a private member of the **Employee class**, use the concept of a **friend class** so that the Payroll class can access and modify the private data member **salary**.

Implement this scenario using proper classes and demonstrate it in the main function.

SOURCECODE:

```
#include <iostream>
using namespace std;
```

```
class Employee {
private:
    string name;
    int id;
    double salary;

public:
    Employee(string n,
int i, double s) {
        name = n;
        id = i;
        salary = s;
```

```

    }

    // Friend Class
    friend class Payroll;
};

class Payroll {
public:
    void
    updateSalary(Employee
    e &e, double
    newSalary) {
        e.salary =
        newSalary; // access
        private member
    }

    void
    display(Employee &e) {
        cout << "Name: "
        << e.name << endl;
        cout << "ID: " <<
        e.id << endl;
        cout << "Salary: "
        << e.salary << endl;
    }
};

int main() {
    Employee
    emp("Faiza", 101,
    50000);
    Payroll p;

    p.display(emp);

```

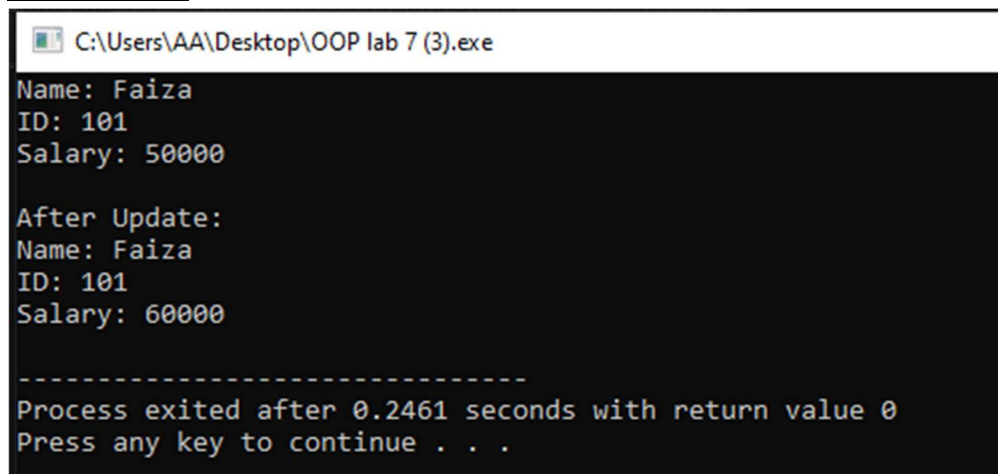


```
p.updateSalary(emp,  
60000);
```

```
cout << "\nAfter  
Update:\n";  
p.display(emp);
```

```
return 0;  
}
```

OUTPUT:



```
C:\Users\AA\Desktop\OOP lab 7 (3).exe  
Name: Faiza  
ID: 101  
Salary: 50000  
  
After Update:  
Name: Faiza  
ID: 101  
Salary: 60000  
  
-----  
Process exited after 0.2461 seconds with return value 0  
Press any key to continue . . .
```

QUESTION: 4

Repeat Question 3, but implement the same scenario using a **friend function** instead of a friend class.

- The **Employee class** should contain private data members:
 - name
 - id
 - designation
 - salary
- A **friend function** should be used to update the salary of the employee.

The friend function should be able to access and modify the private member **salary** of the Employee class.

Demonstrate the working in the main function.

SOURCECODE:

```
#include <iostream>
using namespace std;
```

```
class Employee {
private:
    string name;
    int id;
    double salary;
```

```
public:
    Employee(string n,
int i, double s) {
        name = n;
        id = i;
        salary = s;
    }
```

```
    // Friend Function
Declaration
    friend void
updateSalary(Employee
e &e, double
newSalary);
```

```
    void display() {
        cout << "Name: "
<< name << endl;
        cout << "ID: " << id
<< endl;
        cout << "Salary: "
<< salary << endl;
    }
};
```

```
// Friend Function
Definition (Outside
Class)
void
updateSalary(Employee
e &e, double
newSalary) {
    e.salary = newSalary;
// allowed because it's
friend
}
```

```
int main() {
    Employee
emp("Faiza", 102,
45000);

    emp.display();

    updateSalary(emp,
55000); // calling
friend function

    cout << "\nAfter
Update:\n";
    emp.display();

    return 0;
}
```

OUTPUT:

```
C:\Users\AA\Desktop\OOP lab 7 (4).exe
Name: Faiza
ID: 102
Salary: 45000

After Update:
Name: Faiza
ID: 102
Salary: 55000

-----
Process exited after 0.2529 seconds with return value 0
Press any key to continue . . . _
```

SOLVED EXAMPLES

Example: 1

SOURCE CODE:

```
#include <iostream>
#include <cstdlib>
using namespace std;
```

```
class Array {
    int size;
    int *ptr;

public:
    // Constructor
    Array(int s = 0) {
        size = s;
        ptr = NULL;

        if (size > 0)
            ptr = new
int[size];
```

```
        if (ptr == NULL &&  
size > 0)  
            exit(1);  
    }
```

```
    // Copy Constructor  
    Array(const Array  
&ob) {  
        size = ob.size;
```

```
        if (size > 0) {  
            ptr = new  
int[size];
```

```
            if (ptr == NULL)  
                exit(1);
```

```
            for (int i = 0; i <  
size; i++) {  
                ptr[i] =  
ob.ptr[i];  
            }  
        }  
    }
```

```
    // Indexing Operator  
    int& operator[](int  
index) {  
        if (index >= size) {  
            cout << "Index  
out of bound\n";  
            exit(1);  
        }  
        return ptr[index];  
    }
```

```

// Addition Operator
Array
operator+(Array &ob) {
    if (size == ob.size)
    {
        Array
temp(size);

        for (int i = 0; i <
size; i++) {
            temp[i] =
ptr[i] + ob[i];
        }
        return temp;
    } else {
        cout << "Size
mismatch\n";
        exit(1);
    }
}

```

```

// Dot Product (*)
int operator*(Array
&ob) {
    if (size == ob.size)
    {
        int sum = 0;

        for (int i = 0; i <
size; i++) {
            sum += ptr[i]
* ob[i];
        }
        return sum;
    } else {

```

```
        cout << "Size  
mismatch\n";  
        exit(1);  
    }  
}
```

```
// Destructor  
~Array() {  
    delete[] ptr;  
}  
};
```

```
int main() {  
    Array a1(5), a2(5);
```

```
    for (int i = 0; i < 5;  
i++) {  
        a1[i] = i * 2;  
        a2[i] = i + 3;  
    }
```

```
    cout << "Array 1:\n";  
    for (int i = 0; i < 5;  
i++)  
        cout << a1[i] <<  
endl;
```

```
    cout << "Array 2:\n";  
    for (int i = 0; i < 5;  
i++)  
        cout << a2[i] <<  
endl;
```

```
    Array a3(a1);    //  
Copy constructor
```

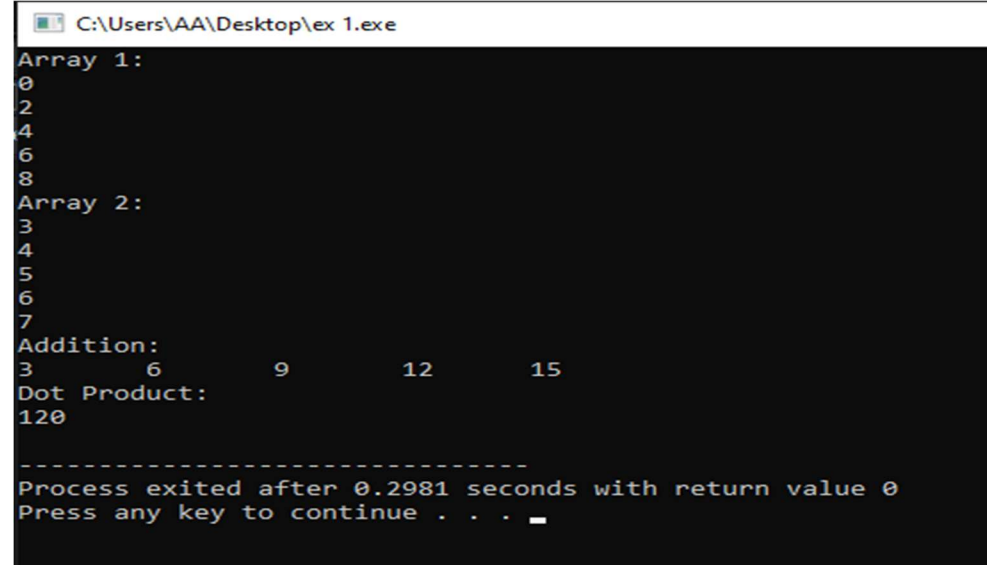
```
    Array a4 = a1 + a2;  
    // Addition
```

```
    cout <<  
    "Addition:\n";  
    for (int i = 0; i < 5;  
    i++)  
        cout << a4[i] <<  
        "\t";
```

```
    cout << "\nDot  
Product:\n";  
    cout << a1 * a2 <<  
    endl;
```

```
    return 0;  
}
```

OUTPUT:



```
C:\Users\AA\Desktop\ex 1.exe  
Array 1:  
0  
2  
4  
6  
8  
Array 2:  
3  
4  
5  
6  
7  
Addition:  
3      6      9      12      15  
Dot Product:  
120  
  
-----  
Process exited after 0.2981 seconds with return value 0  
Press any key to continue . . .
```


Example: 2

SOURCE CODE:

```
#include <iostream>
using namespace std;
```

```
class beta; // Forward
declaration
```

```
class alpha {
private:
    int data;
```

```
public:
    alpha() {
        data = 3;
    }
```

```
    friend int
    frifunc(alpha, beta);
};
```

```
class beta {
private:
    int data;
```

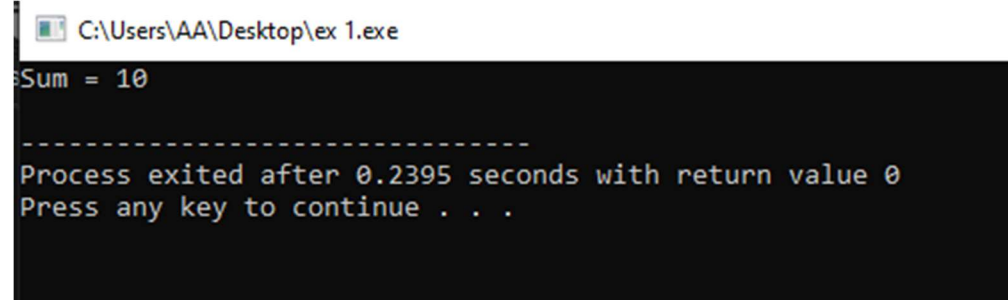
```
public:
    beta() {
        data = 7;
    }
```

```
    friend int
    frifunc(alpha, beta);
};
```

```
// Friend Function
```

```
int frifunc(alpha a,  
beta b) {  
    return a.data +  
    b.data;  
}  
  
int main() {  
    alpha aa;  
    beta bb;  
  
    cout << "Sum = " <<  
    frifunc(aa, bb) << endl;  
  
    return 0;  
}
```

OUTPUT:



The screenshot shows a Windows command prompt window with the title bar "C:\Users\AA\Desktop\ex 1.exe". The output of the program is displayed in the command prompt:

```
Sum = 10  
  
-----  
Process exited after 0.2395 seconds with return value 0  
Press any key to continue . . .
```

Example: 3

SOURCE CODE:

```
#include <iostream>
#include <cstdlib>
using namespace std;

class Array {
    int size;
    int *ptr;

public:
    Array(int s = 0) {
        size = s;
        ptr = NULL;

        if (size > 0)
            ptr = new
int[size];

        if (ptr == NULL &&
size > 0)
            exit(1);
    }

    int& operator[](int
index) {
        if (index >= size) {
            cout << "Index
out of bound\n";
            exit(1);
        }
        return ptr[index];
    }

    // Friend Function
```

```

    friend int
    dotproduct(Array,
    Array);
};

// Friend Function
Definition
int dotproduct(Array
a1, Array a2) {
    if (a1.size == a2.size)
    { // fixed (== instead
of =)
        int sum = 0;

        for (int i = 0; i <
a1.size; i++) {
            sum += a1[i] *
a2[i];
        }
        return sum;
    } else {
        cout << "Size
mismatch error\n";
        exit(1);
    }
}

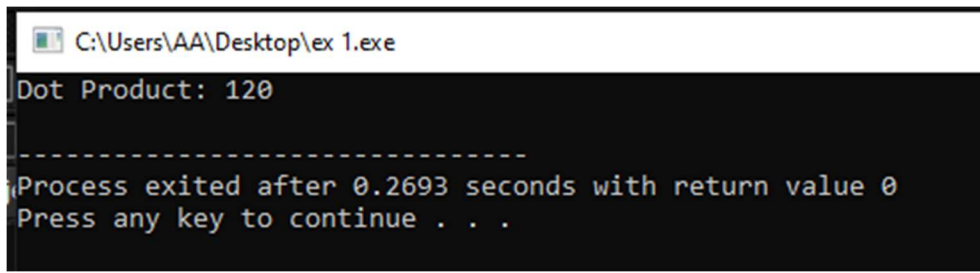
int main() {
    Array a1(5), a2(5);

    for (int i = 0; i < 5;
i++) {
        a1[i] = i * 2;
        a2[i] = i + 3;
    }
}

```

```
    cout << "Dot  
Product: " <<  
dotproduct(a1, a2) <<  
endl;  
  
    return 0;  
}
```

OUTPUT:



```
C:\Users\AA\Desktop\ex 1.exe  
Dot Product: 120  
-----  
Process exited after 0.2693 seconds with return value 0  
Press any key to continue . . .
```

Example: 4

SOURCE CODE:

```
#include <iostream>  
using namespace std;
```

```
class alpha {  
private:  
    int data1;  
  
public:  
    alpha() {  
        data1 = 99;  
    }  
  
    // Friend Class  
    friend class beta;  
};
```

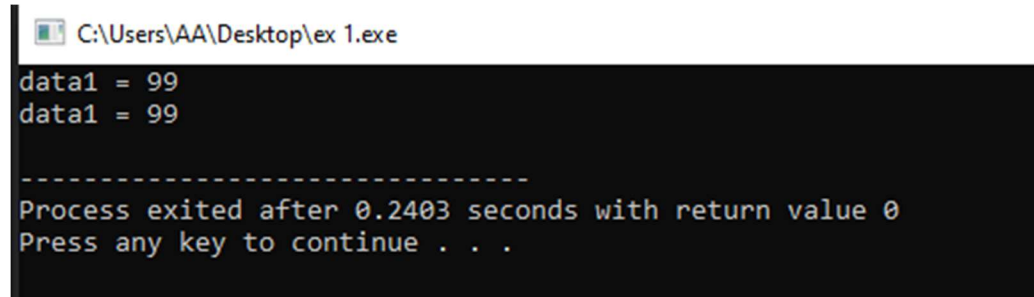
```
class beta {  
public:
```

```
void func1(alpha a) {  
    cout << "data1 = "  
<< a.data1 << endl;  
}
```

```
void func2(alpha a) {  
    cout << "data1 = "  
<< a.data1 << endl;  
}  
};
```

```
int main() {  
    alpha a;  
    beta b;  
  
    b.func1(a);  
    b.func2(a);  
  
    return 0;  
}
```

OUTPUT:



```
C:\Users\AA\Desktop\ex 1.exe  
data1 = 99  
data1 = 99  
  
-----  
Process exited after 0.2403 seconds with return value 0  
Press any key to continue . . .
```